

Chap 2 - Technique Divide and Conquer (D&C)

The Divide and Conquer strategy is detailed and demonstrated over three problems:

- **Binary search algorithm,**
- **Merge sort algorithm,**
- **Karatsuba algorithm**
- **Matrix multiplication.**

1. Introduction à la technique Divide and Conquer (D&C)

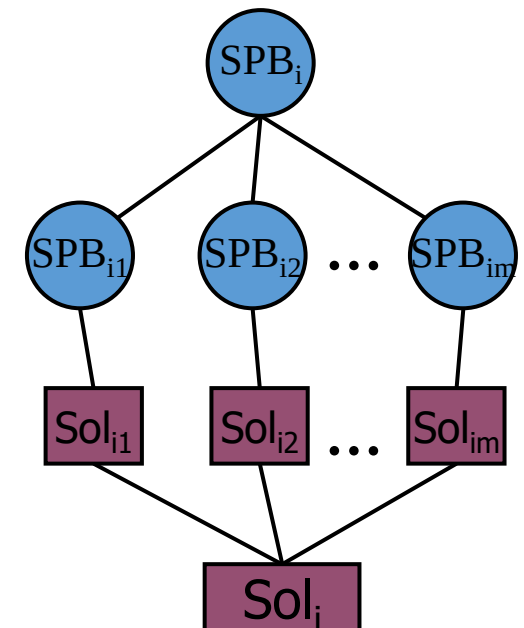
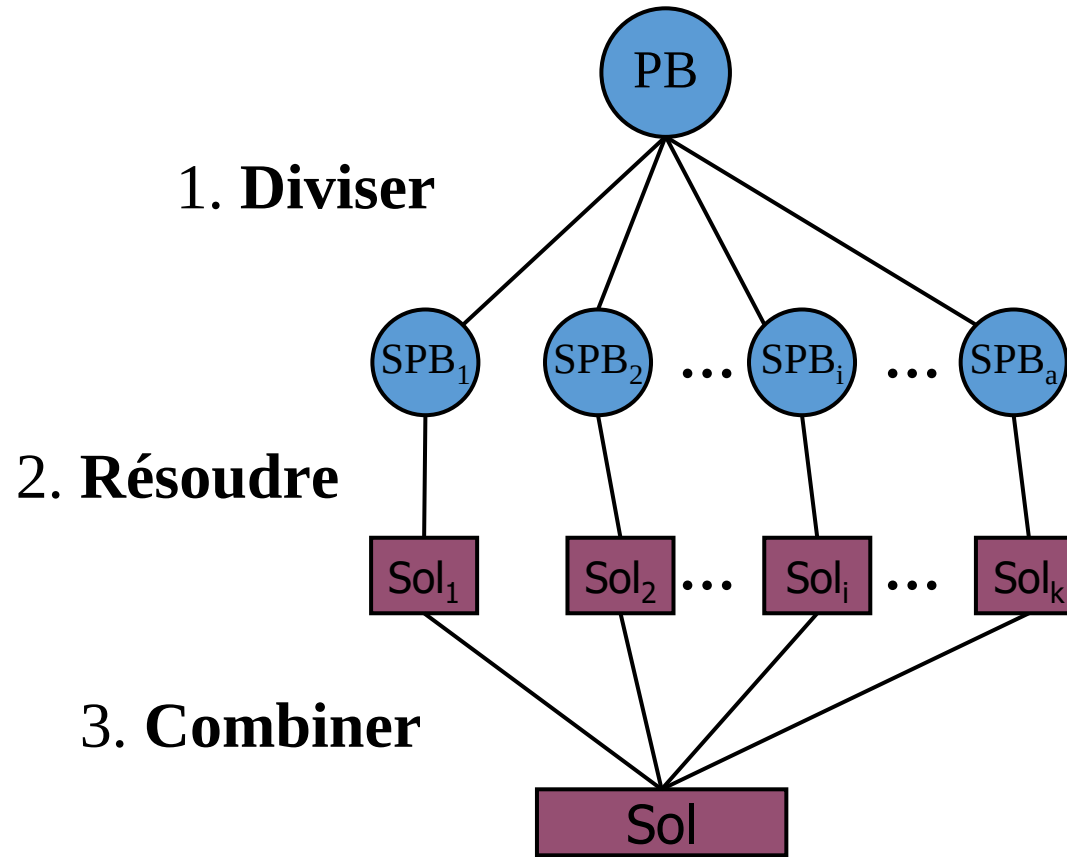
□ Principe

La technique Divide and Conquer (D&C) consiste à ;

- **Décomposer un problème en sous-problèmes plus petits,**
- **Résoudre les sous problèmes,**
- **Combiner les solutions de ces sous-problèmes pour résoudre le problème initial.**

Les **sous-problèmes doivent être de même nature** que le problème global et sont résolus récursivement.

□ Etapes d'un algorithme D&C



❑ algorithme générique D&C

Algorithm Div_and_Conq(P, n):

#Input: Problem P of size n

#Output: Solution to problem P

#1. Base Case:

if n <= threshold: **return** baseSol(P,n)

#2. Divide:

subPbs= **Divide**(P,n)#[(P1,n1),(P2,n2), ..., (Pk,nk)]

#3. Conquer:

SubSols =[**Div_and_Conq**((Pi,ni) for (Pi,ni) in subPbs]

#4. Combine:

return **Combine**(SubSols)

□ Complexité

L'analyse des algorithmes de type Divide and Conquer peut être faite en utilisant une **équation de récurrence**. Elle est souvent représentée par la relation de récurrence suivante :

$$\begin{cases} T(n_0) = T_0, & n_0 \leq \textit{threshold} \\ T(n) = k T\left(\frac{n}{b}\right) + O(f(n)), \forall n > n_0 \end{cases}$$

Où

- $T(n)$: Complexité du problème de taille n .
- k : Nombre de sous-problèmes.
- b : Coefficient de réduction de taille de chaque sous-problème (n/b).
- $f(n)$: Coût de la division et combinaison.
- T_0 : Coût de la résolution des sous-problèmes les plus petits (cas de base)

□ Analyse de la complexité avec le théorème de Maître

La complexité d'un algorithme de type D&C dépend de la manière dont les sous-problèmes sont décomposés et combinés. Le **théorème de Maître (Master Theorem)** fournit un cadre pour analyser cette complexité. Selon la manière dont les sous-problèmes sont divisés, on obtient trois principaux cas :

– **Cas 1 : $f(n) = O(n^c)$ avec $c < \log_b k$**

Complexité : $O(n^{\log_b k})$

=> Le coût de résolution des sous-problèmes domine.

– **Cas 2 : $f(n) = O(n^{\log_b k})$**

Complexité : $O(n^{\log_b k} \log n)$

=> Le coût de résolution des sous-problèmes et le coût de combinaison/division sont équivalents.

– **Cas 3 : $f(n) = O(n^c)$ avec $c > \log_b k$**

Complexité : $O(n^c)$

=> Le coût de la combinaison/division domine

2. Exemples d'algorithmes Divide and Conquer

□ Algorithme Binary Search (Recherche dichotomique)

divise un tableau trié en deux moitiés et recherche la valeur dans la moitié pertinente.

- Complexité :

$$O(\log n)$$

- Relation de récurrence :

$$T(n) = T(n/2) + O(1)$$

- Pseudo code

Algorithm BinarySearch(A, target, low, high):

if low > high then return -1

mid = (low + high) // 2

if A[mid] == target then return mid

else if A[mid] > target then

return BinarySearch(A, target, low, mid-1)

else

return BinarySearch(A, target, mid+1, high)

□ Algorithme Merge Sort (Tri fusion)

Merge Sort divise un tableau en sous-tableaux, les trie récursivement, puis les fusionne pour former un tableau global trié.

- **Complexité :**

$$O(n \log n)$$

- **Relation de récurrence :**

$$T(n) = 2T(n/2) + O(n)$$

- **Pseudocode - Merge Sort**

Algorithm MergeSort(A, l, h):

if l >= h **then return**

m = (l + h) // 2

MergeSort(A, l, m)

MergeSort(A, m + 1, h)

Merge(A, l, m, h)

□ Algorithme de Karatsuba (Multiplication)

L'algorithme de Karatsuba multiplie deux grands nombres x et y en les divisant en plusieurs sous-nombres, puis combine les produits partiels. Il se base sur une approche de type **divide and conquer** pour réduire la complexité.

▪ Etapes du calcul classique :

Soient deux nombres x et y de taille n , qu'on veut multiplier. On peut les représenter comme des polynômes :

$$x = x_1 \cdot 10^m + x_0$$

$$y = y_1 \cdot 10^m + y_0$$

Où x_1 et x_0 sont les parties "haute" et "basse" du nombre x , et de même pour y_1 et y_0 , et m est approximativement égal à $n/2$.

Ainsi, le produit $x \cdot y$ peut être réécrit comme :

$$x \cdot y = (x_1 \cdot 10^m + x_0) \cdot (y_1 \cdot 10^m + y_0)$$

$$x \cdot y = x_1 \cdot y_1 \cdot 10^{2m} + (x_1 \cdot y_0 + x_0 \cdot y_1) \cdot 10^m + x_0 \cdot y_0$$

Dans la méthode classique, cela nécessite 4 multiplications (les termes $x_1 \cdot y_1$, $x_0 \cdot y_0$, $x_1 \cdot y_0$, et $x_0 \cdot y_1$).

▪ Etapes de calcul de Karatsuba :

Astuce de Karatsuba : Réduire les 4 multiplications à seulement 3 en suivant les étapes :

1. Calcul des trois multiplications suivantes :

$$A = x_1 \cdot y_1$$

$$B = x_0 \cdot y_0$$

$$C = (x_1 + x_0) \cdot (y_1 + y_0)$$

2. Ensuite, on remarque que :

$$C = A + B + (x_1 \cdot y_0 + x_0 \cdot y_1)$$

$$\text{Donc, } x_1 \cdot y_0 + x_0 \cdot y_1 = C - A - B$$

3. Combinaison des résultats : Le produit final est donné par :

$$x \cdot y = A \cdot 10^{2m} + (C - A - B) \cdot 10^m + B$$

- Complexité :

$$O(n^{\log_2 3}) = O(n^{1.58})$$

- Relation de récurrence :

$$T(n) = 3T(n/2) + O(n)$$

- Pseudo code

Algorithm Karatsuba(x, y):

if x or y < 10 **then return** x * y

m = max(len(x), len(y)) // 2

x1, x0 = split(x, m)

y1, y0 = split(y, m)

A = Karatsuba(x1, y1)

B = Karatsuba(x0, y0)

C = Karatsuba((x1 + x0), (y1 + y0))

return A * 10^(2*m) + (C-A-B) * 10^m + B

❑ Algorithme classique de Multiplication de matrices (approche Force brute)

La multiplication de deux matrices carrées A et B de taille $n \times n$ est définie comme suit :

$$C[i][j] = \sum_{k=1}^n A[i][k] * B[k][j]$$

▪ Complexité :

La complexité de cette **approche brute-force** est $O(n^3)$, car on doit réaliser n^2 calculs d'éléments, chacun nécessitant n opérations de multiplication et d'addition.

▪ Pseudo code

Algorithm MatriceMult(A, B):

for i **in** $0..n-1$:

for j **in** $0..n-1$:

$C[i][j] = 0$

for k **in** $0..n-1$:

$C[i][j] = C[i][j] + A[i][k] * B[k][j]$

Return C

□ Algorithme D&C de Multiplication de matrices

▪ Etape 1 : Diviser

L'idée est de diviser les matrices A , B et C en sous-matrices de taille $n/2 \times n/2$. Pour cela, on découpe chacune de ces matrices en quatre blocs de taille égale :

$$A = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix}, \quad B = \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix}, \quad C = \begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix}$$

▪ Etape 2 : Résoudre les sous-problèmes

On applique la règle de multiplication des matrices blocs. Les éléments de la matrice C se calculent comme suit :

$$C_{11} = A_{11} \times B_{11} + A_{12} \times B_{21}$$

$$C_{12} = A_{11} \times B_{12} + A_{12} \times B_{22}$$

$$C_{21} = A_{21} \times B_{11} + A_{22} \times B_{21}$$

$$C_{22} = A_{21} \times B_{12} + A_{22} \times B_{22}$$

▪ Etape 3 : Combiner :

Pour chaque sous-problème, on combine les résultats des sous-matrices pour obtenir la matrice complète C .

▪ Complexité

En termes de récursivité, nous avons huit sous-problèmes de multiplication de matrices de taille $n/2 \times n/2$, et chacun nécessite ensuite $O(n^2)$ opérations pour combiner les sous-matrices. Le temps de calcul total peut être exprimé par la relation de récurrence suivante :

$$T(n) = 8T\left(\frac{n}{2}\right) + O(n^2)$$

En résolvant cette équation de récurrence (en utilisant le théorème de maître par exemple), on trouve que la complexité totale est :

$$T(n) = O(n^3)$$

▪ Pseudo code

Algorithm MatrixMultiply(A, B):

if size of A is 1:

return A * B # multiplication of individual elements

else:

Divide A into A11, A12, A21, A22

Divide B into B11, B12, B21, B22

C11 = MatrixMultiply(A11, B11) + MatrixMultiply(A12, B21)

C12 = MatrixMultiply(A11, B12) + MatrixMultiply(A12, B22)

C21 = MatrixMultiply(A21, B11) + MatrixMultiply(A22, B21)

C22 = MatrixMultiply(A21, B12) + MatrixMultiply(A22, B22)

Combine C11, C12, C21, C22 into C

return C

3. Conclusion et avantages de D&C

□ Avantages :

- **Modularité** : Le problème est résolu par étapes, chaque étape étant une version plus simple du problème initial.
- **Efficacité** : Pour de nombreux problèmes, cette méthode réduit considérablement la complexité.
- **Parallélisation** : Les sous-problèmes étant souvent indépendants, ils peuvent être résolus en parallèle.

□ Inconvénients :

- **Surcharge de mémoire** : Les appels récursifs peuvent entraîner une consommation importante de mémoire.
- **Combinaison coûteuse** : L'étape de combinaison peut parfois être plus coûteuse que la division.

4. Algorithme Strassen de Multiplication de matrices (Devoir libre)

Objectif :

Comprendre et appliquer l'algorithme de Strassen pour la multiplication de matrices en utilisant la stratégie Divide & Conquer.

Enoncé :

La multiplication de matrices est une opération fondamentale en algèbre linéaire avec des applications variées en calcul scientifique, en traitement d'images, en apprentissage automatique, et bien d'autres domaines. L'algorithme de multiplication classique de matrices carrées a une complexité $O(n^3)$, mais il existe des approches plus efficaces qui permettent de réduire cette complexité, en l'occurrence celle de Strassen.

1. Faites des recherches sur l'algorithme de Strassen pour la multiplication de matrices.
2. Expliquez le principe de cet algorithme et en quoi il diffère de la méthode classique.
3. Proposez les étapes détaillées de l'algorithme de Strassen pour la multiplication de deux matrices carrées de taille $n \times n$.
4. Indiquez les cas où l'algorithme de Strassen est plus efficace que la méthode classique.
5. Formulez l'équation de récurrence qui décrit la complexité de l'algorithme de Strassen.
6. Résolvez cette équation pour obtenir la complexité temporelle de Strassen en fonction de n .
7. Implémentez l'algorithme de Strassen en C ou en Python pour multiplier deux matrices carrées de taille $n=2^p$

5. Codage Python des exemples d'algorithmes Divide and Conquer

❑ Algorithme Binary Search (Recherche dichotomique) _ basé sur les listes

```
def binary_search(arr, target, low, high):
    if low > high: # Cas de base : si l'intervalle est invalide
        return -1 # Élément non trouvé
    # Étape 1 : Calculer l'indice du milieu
    mid = (low + high) // 2
    # Étape 2 : Si l'élément est trouvé au milieu
    if arr[mid] == target:
        return mid
    # Étape 3 : Diviser et conquérir
    elif arr[mid] > target:
        return binary_search(arr, target, low, mid - 1)
    else:
        return binary_search(arr, target, mid + 1, high)

if __name__ == "__main__": # Exemple d'utilisation
    Target=7
    arr = [1, 3, 5, 7, 9, 11, 13, 15, 17, 19] # Tableau trié
    # Appel de la fonction avec les bornes low = 0 et high = longueur du tableau - 1
    result = binary_search(arr, target, 0, len(arr) - 1)
    if result != -1:
        print(f"L'élément {target} se trouve à l'index {result}.")
    else:
        print(f"L'élément {target} n'a pas été trouvé dans le tableau.")
```

❑ Algorithme Binary Search (Recherche dichotomique) _ basé sur les tableau numpy

```
import numpy as np

def binary_search(arr, target, low, high):
    if low > high: # Cas de base : si l'intervalle est invalide
        return -1 # Élément non trouvé
    # Étape 1 : Calculer l'indice du milieu
    mid = (low + high) // 2
    # Étape 2 : Si l'élément est trouvé au milieu
    if arr[mid] == target:
        return mid
    # Étape 3 : Diviser et conquérir
    elif arr[mid] > target:
        return binary_search(arr, target, low, mid - 1)
    else:
        return binary_search(arr, target, mid + 1, high)

if __name__ == "__main__": # Exemple d'utilisation
    Target=7
    arr = np.array([1, 3, 5, 7, 9, 11, 13, 15, 17, 19]) # Tableau trié
    # Appel de la fonction avec les bornes low = 0 et high = longueur du tableau - 1
    result = binary_search(arr, target, 0, len(arr) - 1)
    if result != -1:
        print(f"L'élément {target} se trouve à l'index {result}.")
    else:
        print(f"L'élément {target} n'a pas été trouvé dans le tableau.")
```

❑ Algorithme Merge Sort (Tri fusion)

```
def merge_sort(A, l, h):  
    if l >= h: return  
    mid = (l + h) // 2 # Calcul de la position du milieu  
    merge_sort(A, l, mid) # Tri de la première moitié (de l à mid)  
    merge_sort(A, mid + 1, h) # Tri de la deuxième moitié (de mid + 1 à h)  
    merge(A, l, mid, h) # Fusionner les deux sous-tableaux triés  
  
def merge(A, l, mid, h):  
    # Sous-tableaux à fusionner  
    left, right = A[l:mid + 1], A[mid + 1:h + 1]  
    i, j, k = 0, 0, l  
    while i < len(left) and j < len(right): # Fusionner en fonction des valeurs des sous-tableaux  
        if left[i] <= right[j]:  
            A[k], i = left[i], i + 1  
        else:  
            A[k], j = right[j], j + 1  
        k += 1  
    while i < len(left): # Ajouter les éléments restants du sous-tableau gauche (s'il y en a)  
        A[k], i, k = left[i], i + 1, k + 1  
    while j < len(right): # Ajouter les éléments restants du sous-tableau droit (s'il y en a)  
        A[k], j, k = right[j], j + 1, k + 1  
  
if __name__ == "__main__": # Exemple d'utilisation  
    A = [38, 27, 43, 3, 9, 82, 10]  
    merge_sort(A, 0, len(A) - 1)  
    print(f"Tableau trié : {A}")
```

❑ Algorithme de Karatsuba (Multiplication)

```
def split(number, m): # Divise un nombre en deux parties.
    str_num = str(number)
    high = str_num[:-m] or '0' # Partie haute
    low = str_num[-m:] # Partie basse
    return int(high), int(low)

def karatsuba(x, y): # Multiplie x et y en utilisant l'algorithme de Karatsuba.
    if x < 10 or y < 10: return x * y # Cas de base
    m = max(len(str(x)), len(str(y))) // 2 # Trouver la longueur maximum

    x1, x0 = split(x, m) # Diviser x en deux moitiés
    y1, y0 = split(y, m) # Diviser y en deux moitiés

    # Appels récurifs
    A = karatsuba(x1, y1) # Partie haute
    B = karatsuba(x0, y0) # Partie basse
    C = karatsuba(x1 + x0, y1 + y0) # Partie croisée

    return (A * 10**(2 * m)) + ((C - A - B) * 10**m) + B # Calcul de la multiplication

if __name__ == "__main__": # Exemple d'utilisation
    result = karatsuba(1234, 5678)
    print("Le produit de 1234 et 5678 est :", result)
```

❑ Algorithme classique de Multiplication de matrices (approche Force brute)

```
def matrix_mult(A, B):#Multiplie deux matrices A et B.
    # Vérifier que les matrices peuvent être multipliées
    if len(A[0]) != len(B):
        raise ValueError("Le nombre de colonnes de A doit être égal au nombre de lignes de B.")

    # Initialiser la matrice résultat C
    n,m = len(A), len(B[0])
    C = [[0 for _ in range(m)] for _ in range(n)]
    for i in range(n): # Multiplication des matrices
        for j in range(m):
            C[i][j] = 0 # Initialiser C[i][j]
            for k in range(len(B)):
                C[i][j] += A[i][k] * B[k][j]

    return C

if __name__ == "__main__": # Exemple d'utilisation
    A = [
        [1, 2, 3],
        [4, 5, 6],]

    B = [
        [7, 8],
        [9, 10],
        [11, 12],]

    result = matrix_mult(A, B)
    print("Le produit des matrices A et B est :")
    for row in result:      print(row)
```

■ Algorithme D&C de Multiplication de matrices

```
def matrix_multiply(A, B):
    """Multiplie deux matrices A et B en utilisant l'algorithme D&C."""
    # Vérifier que les matrices peuvent être multipliées
    if len(A) != len(A[0]) or len(B) != len(B[0]):
        raise ValueError("Les matrices doivent être carrées.")

    n = len(A) # Supposer que A et B sont des matrices n x n

    # Cas de base : si la taille de A est 1, retourner le produit des éléments
    if n == 1:
        return [[A[0][0] * B[0][0]]]

    # Diviser les matrices A et B
    mid = n // 2

    A11 = [[A[i][j] for j in range(mid)] for i in range(mid)]
    A12 = [[A[i][j] for j in range(mid, n)] for i in range(mid)]
    A21 = [[A[i][j] for j in range(mid)] for i in range(mid, n)]
    A22 = [[A[i][j] for j in range(mid, n)] for i in range(mid, n)]

    B11 = [[B[i][j] for j in range(mid)] for i in range(mid)]
    B12 = [[B[i][j] for j in range(mid, n)] for i in range(mid)]
    B21 = [[B[i][j] for j in range(mid)] for i in range(mid, n)]
    B22 = [[B[i][j] for j in range(mid, n)] for i in range(mid, n)]

    # Appeler récursivement la multiplication de matrices
    C11 = add_matrices(matrix_multiply(A11, B11), matrix_multiply(A12, B21))
    C12 = add_matrices(matrix_multiply(A11, B12), matrix_multiply(A12, B22))
    C21 = add_matrices(matrix_multiply(A21, B11), matrix_multiply(A22, B21))
    C22 = add_matrices(matrix_multiply(A21, B12), matrix_multiply(A22, B22))

    # Combiner les sous-matrices C11, C12, C21, C22 en une matrice C
    C = combine_matrices(C11, C12, C21, C22)
    return C
```

Chap2. Technique Divide and Conquer (D&C)

```
def add_matrices(A, B):
    """Ajoute deux matrices de même taille."""
    return [[A[i][j] + B[i][j] for j in range(len(A[0]))] for i in range(len(A))]

def combine_matrices(C11, C12, C21, C22):
    """Combine les sous-matrices en une matrice complète."""
    top = [C11[i] + C12[i] for i in range(len(C11))]
    bottom = [C21[i] + C22[i] for i in range(len(C21))]
    return top + bottom

if __name__ == "__main__": # Exemple d'utilisation
    A = [
        [1, 2, 3, 4],
        [5, 6, 7, 8],
        [9, 10, 11, 12],
        [13, 14, 15, 16]
    ]

    B = [
        [1, 2, 3, 4],
        [5, 6, 7, 8],
        [9, 10, 11, 12],
        [13, 14, 15, 16]
    ]

    result = matrix_multiply(A, B)
    print("Le produit des matrices A et B est :")
    for row in result:
        print(row)
```